

Planet Merge: Performance Guide

What is expensive, what we already fixed, what still lags, and what Nikola can do about it. July 2026.

TL;DR. The game has two jobs every frame: **simulate** (Matter.js physics) and **paint** (canvas drawing). Painting used to be the bottleneck and is now mostly solved by pre-baking every SVG into a bitmap. Today the real cost is **physics on a crowded late-game board**, and the single biggest thing you can do in Illustrator is **simplify the invisible silhouette paths inside planet_saturn.svg and planet_sun.svg**. Reducing anchor points on the other artwork helps loading, not framerate, because after load the game never touches the SVGs again.

1. The frame budget

At 60 Hz the game has **16.7 ms** per frame. In that window it must run up to 4 physics substeps (fixed 8 ms steps fed by real elapsed time) and then repaint the whole 840 x 1160 canvas. A frame that takes longer is a visible stutter. A phone has roughly a quarter of a desktop's CPU and GPU budget, which is why everything below matters 4x more on mobile.

2. The expensive things, ranked

#1 Physics collision solving on a full board

Matter.js has no true circles. Every "round" planet is really a polygon with **64 sides** (32 on phones), and Saturn and the Sun are **compound bodies**: their outline is traced from the SVG silhouette (one sample every 12 px), and because the result is concave, poly-decomp splits it into many small convex pieces. Every piece is its own collider. When two Suns rest against each other, the engine is not solving one contact, it is solving dozens of polygon pairs, 125 times per second.

- Cost grows with how full the board is. An early board with 8 small planets is nearly free; a late board with 25+ bodies including Suns, Saturns and Jupiters is the worst case in the whole game.
- Merge cascades are spikes: every merge wakes all sleeping bodies (needed, see the CLAUDE.md), so during a chain nothing sleeps and the solver runs hot.
- Each time a Saturn or Sun spawns, its silhouette is decomposed again from scratch (`Bodies.fromVertices` in physics.js). On a slow phone that is a small hitch right in the middle of a cascade.

#2 Painting pixels (fill rate)

Every frame repaints the full container gradient, ~70 twinkling stars, every planet sprite (a Sun is a 410 x 410 px draw), the ship, its beam, and the big sky score. Behind the game, a separate full-viewport starfield canvas paints at ~30 fps. On desktop GPUs this is comfortable; on weak phones it is fill-rate bound, which is why mobile renders the canvas at 0.7 scale (roughly half the pixels).

- The ship beam builds 4 gradient objects per frame and uses `lighter` blending (an extra read-modify-write pass). Skipped on mobile.
- The sky score + moving ship shadow redraws an offscreen 840 x 262 band every frame on desktop. Mobile uses a cached bitmap instead.

#3 SVG rasterization (solved, but the rule must hold)

This was historically the #1 lag source: browsers re-rasterize vector images on almost every `drawImage`, and the game was doing it for every planet, every frame. Since the bitmap-baking rework it happens exactly once per asset at load. **The rule: never draw an SVG-backed image inside the game loop.** Bake it, then blit the bitmap. One-off UI (legend, perk tiles, start screen) may use SVGs directly.

#4 Small steady costs

- Per-frame allocations: the live-shapes list is rebuilt every substep and every paint, and the beam allocates gradients. Individually tiny, together they feed the garbage collector.
- Load time: ~45 small SVG fetches (12 bodies, 30 face overlays, ship skins), the Fredoka font, Matter.js and poly-decomp from CDNs, plus ~250 KB of preloaded sound effects. The three voice-over clips (1.9 MB total) load only when a perk's play button is pressed, so they cost nothing at start.

3. What is already in place

Problem	Shipped fix
SVG re-rasterized every frame (worst offender)	Every planet SVG baked once into an offscreen bitmap; body + face pre-combined so a planet with a mood is still one <code>drawImage</code> .
Canvas filters force a full pixel pass	No filters in the loop. The ship's shadow on the score is a pre-baked black silhouette, not a brightness filter.
Phones are fill-rate and CPU bound	Mobile perf mode (viewport ≤ 700 px or coarse pointer): canvas at 0.7 render scale, beam skipped, score drawn from a cached bitmap, 32-sided colliders, autopilot hidden.
Game speed tied to monitor refresh; fast bodies tunneling	Fixed 8 ms physics steps from an accumulator, capped at 32 ms per frame so a hitch slows the sim instead of snowballing.
Contact solver never resting	Sleeping enabled; wakes are targeted and time-boxed (a wake window of a couple of seconds, then bodies settle again).
Background starfield ate the frame budget on big monitors	Page starfield renders at plain resolution (no high-DPI supersampling), ~30 fps, with the nebula baked once.
Fast autopilot on weaker machines	Lite render mode draws half the in-canvas stars and enforces drop-rate floors.
Audio spam during cascades	Impact sounds capped at one per kind per physics tick; pooled clips so pops can overlap without restarting.
Death replay bookkeeping	Position history records only while a planet is near the rim, so calm play pays nothing.

4. What still lags, honestly

- **Late-game boards.** Several Suns/Saturns resting on each other multiply compound-vs-compound contacts. This is the main remaining cost and it is physics, not painting.
- **Cascade spikes.** A long chain wakes everything, spawns bodies (with per-spawn decomposition for Saturn/Sun), separates overlaps, and fires effects in the same few frames.
- **Old/cheap phones.** Even at 0.7 scale, 25 polygon bodies plus full-canvas paint is close to the budget; the earthquake level is the stress peak.

5. What you can do (the artwork checklist)

The one change that buys real FPS: simplify the silhouette paths. Only `planet_saturn.svg` and `planet_sun.svg` have `outline: true`, and only their `<path id="silhouette...">` elements become physics. The visible artwork can stay as detailed as you like; the silhouette is invisible and exists purely for collision.

- **Sun:** the spiky ray outline decomposes into dozens of convex slivers. Give the silhouette far fewer, fatter, rounder bumps, or make it a plain circle (gameplay barely changes: rays were never something players aim at). Every removed spike is removed solver work on every physics step for every Sun on the board.
- **Saturn:** keep it to two smooth subpaths, the ball and one clean ring loop. No notches, no tiny curls.
- Rule of thumb: if a bump on the outline is too small to feel with a finger on a toy, the collider does not need it. The sampler walks the path every 12 px, so detail finer than ~12 px only adds vertices without adding shape.
- Keep the conventions or the loader breaks: the id must still start with `silhouette`, the path must stay inside the same `viewBox`, and separate pieces should be separate subpaths (tiny accidental subpaths under 5% of the biggest are auto-dropped).
- Verification is built in: the console prints `silhouette ready for Sun: N piece(s), M verts total` at boot. Your goal is simply to make N and M smaller. Before/after screenshots of that line are the whole benchmark.

Reducing anchor points on the rest of the art

Honest expectation: fewer anchor points on normal planet art improves **download size, load time and bake time**, not steady-state FPS. After load, the game only ever copies bitmaps; a 500-anchor Jupiter and a 50-anchor Jupiter paint at identical speed. Still worth doing, just don't expect the late-game stutter to change.

If you do a simplification pass, start with the heaviest files (sizes today):

File	Size	Note
<code>planet_jupiter_body.svg</code>	19.2 KB	Baked for gameplay at every load
<code>planet_saturn.svg</code>	15.9 KB	Also carries the silhouette (see above)
<code>planet_jupiter.svg</code>	15.4 KB	Blog cards + legend/perk icon
<code>planet_earth.svg</code>	11.5 KB	Blog cards + perk icon
<code>planet_moon_body.svg</code>	10.6 KB	Baked for gameplay at every load

Illustrator/export tips: Object > Path > Simplify on busy paths, export with 1 decimal of precision, no metadata, no hidden layers, never embed raster images inside the SVG. Running the files through **SVGO** afterwards is safe as long as ids and the viewBox are preserved (`cleanupIds` must not rename `silhouette*`).

Housekeeping (repo and deploy size only, zero FPS impact)

These files sit in `/public` so they ship with every deploy, but the game never requests them, so they cause no lag. Deleting is pure hygiene:

Path	Size	Status
<code>assets/images/old-planets/</code> (with <code>all-planets.ai</code>)	1.69 MB	Unreferenced, safe to delete
<code>assets/images/ship-skins/</code> (source art + <code>all_ships.ai</code>)	125 KB	Unreferenced (the game uses the root <code>ship-container_*.svg</code> files)
<code>game.js.bak</code>	90 KB	Leftover backup
<code>assets/images/all-planets.svg</code>	68 KB	Unreferenced
<code>planet_moon-01.svg</code> , <code>planet_moon1.svg</code> , <code>planet_sad.svg</code> , <code>player_container_placeholder2.svg</code>	19 KB	Unreferenced
<code>A71CB6F3.jpg</code> , <code>main-character.svg</code>	59 KB	Unreferenced today; keep them if they are work-in-progress for a future feature

Do not delete the root `planet_*.svg` files (moon, mars, earth, jupiter, and the rest): the blog post's card grid in `posts.json` links to all of them, and the game itself uses `planet_star.svg` , `planet_saturn.svg` and `planet_sun.svg` directly.

6. How to measure any change

1. Run the site (`npm run dev`) and open `http://localhost:3000/games/planet-merge/play.html?dev=1` .
2. Build a worst case fast: dev panel > Drop > Saturn (or Sun), turn on Auto-Drop, raise Speed. The board fills with the heaviest colliders.
3. Open Chrome DevTools > Performance, set CPU throttling to 4x or 6x to imitate a phone, record ~10 seconds, and look at the frame track: green flat = fine, tall spiky bars = over budget. The biggest blocks will be labelled either `Engine.update` (physics) or paint calls.
4. For silhouette work, compare the boot console line `silhouette ready for ...: N piece(s), M verts total` before and after.

7. Code-side ideas for later (not your job)

- Decompose Saturn/Sun colliders once at boot and clone the parts on spawn, instead of re-running poly-decomp per spawn.
- Scale circle collider sides with radius: a Star does not need 64 sides, only the big planets do.
- Bake the ship beam into a per-skin bitmap (it never changes shape) to drop the 4 per-frame gradients.
- Reuse one scratch array for the live-shapes list to cut GC churn.
- If the silhouettes get smoother, raise the sample spacing from 12 px to ~16 px for another free vertex cut.

